



Plateforme IoT

IndustriTech

DOCUMENTATION TECHNIQUE

v2.1 — 14 mars 2026

• Production

43

CAPTEURS

14

MACHINES

3

ZONES

6

SERVICES

Resume executif

La plateforme **IndustriTech IoT** est un systeme de supervision industrielle en temps reel deploye via Docker Compose. Elle collecte, traite et visualise les donnees de **43 capteurs** repartis sur **14 machines** dans 3 zones d'une usine, via la chaine MQTT → Node-RED → InfluxDB → Webapp. La webapp integre une interface moderne (themes sombre/clair, 6 couleurs) avec vue 3D, cartographie SVG, IA predictive (Z-score, regression lineaire), comparaison multi-machines, alertes push, export PDF/CSV et mode kiosque.

Service	Port(s)	URL
Webapp (supervision)	8080	http://localhost:8080
Node-RED	1880	http://localhost:1880
InfluxDB	8086	http://localhost:8086
Grafana	3000	http://localhost:3000
MQTT	1883 / 8883 / 9001	Docker interne / TLS / Web-Socket

1. Table des matieres

Table des matières

1. Table des matieres	4
2. Architecture generale	9
2.1. Vue d'ensemble	10
2.2. Flux de donnees	10
2.3. Points d'accès	10
3. Webapp de supervision – IndustriTech	13
3.1. Description generale	14
3.2. Pages et fonctionnalites	14
3.3. Architecture backend (server.js)	16
4. Composants de la plateforme	19
4.1. Mosquitto – Broker MQTT	20
4.2. Node-RED – Traitement des flux	21
4.3. InfluxDB 2 – Base de donnees temporelle	21
5. Simulateur de capteurs	24
5.1. Description	25
5.2. Zone Salle Serveur – 10 capteurs, 4 machines	25
5.3. Zone Atelier – 19 capteurs, 6 machines	25
5.4. Zone Local Electrique – 14 capteurs, 4 machines	26
5.5. Format du payload MQTT	26
6. Securite	29
6.1. Certificats TLS	30
6.2. Authentification	30
6.3. Reseau Docker	30
7. Docker Compose	32
8. Makefile – Reference des commandes	36
9. Tests d'integration	39
10. Exploitation et maintenance	42
10.1. Commandes de diagnostic	43
10.2. Procedures de maintenance	43
10.3. Guide de depannage	43
11. DevSecOps	46
11.1. Pipeline CI/CD securise	47
11.2. Analyse des dependances (SCA)	48
11.3. SAST – Analyse statique du code	49
11.4. Scan d'image Docker (Trivy)	50
11.5. Conformite OWASP IoT Top 10	50
11.6. Gestion des secrets	51
11.7. Matrice de menaces (STRIDE)	52
11.8. Surface d'attaque reseau	53
12. Annexes	55
12.1. Recapitulatif des versions	56
12.2. Evolutions possibles	56

2. Architecture generale

2.1. Vue d'ensemble

La plateforme repose sur six services Docker communiquant via un réseau bridge privé `iot-network` :

Service	Image Docker	Role
Mosquitto	eclipse-mosquitto	Broker MQTT central – 3 listeners (1883, 8883, 9001)
Sensor Simulator	python:3.11-slim	Emulation de 43 capteurs sur 14 machines
Node-RED	node-red/node-red:3.1	Routage MQTT -> InfluxDB + règles métier
InfluxDB	influxdb:2.7	Base de données séries temporelles (bucket <code>iot_data</code>)
Grafana	grafana/grafana	Dashboards de visualisation avancés
Webapp	node:20-alpine	Interface supervision temps réel + IA – port 8080

2.2. Flux de données

```
[Simulateur Python]
  | MQTT publish factory/{zone}/{machine}/{capteur}
  v
[Mosquitto :1883]
  | subscribe factory/#
  v
[Node-RED] --format LineProtocol--> [InfluxDB :8086]
  |                               ^
  | WebSocket MQTT :9001         | API Flux
  v                               |
[Webapp :8080] <-- /api/sensors/latest, /api/history --+
```

2.3. Points d'accès

Port	Service	Protocole	Usage
------	---------	-----------	-------

1880	Node-RED	HTTP	Interface flows
1883	Mosquitto	MQTT	Broker interne Docker
3000	Grafana	HTTP	Dashboards
8080	Webapp	HTTP	Supervision temps reel
8086	InfluxDB	HTTP	API REST + UI
8883	Mosquitto	MQTT/TLS	Clients externes securises
9001	Mosquitto	WebSocket	Connexion MQTT depuis la webapp

3. Webapp de supervision – IndustriTech

3.1. Description generale

La webapp (webapp/) est une application Node.js/Express servant une SPA vanilla JS avec Tailwind CSS. Elle se connecte directement a Mosquitto via WebSocket (port 9001) pour un affichage en temps reel, et interroge son propre backend (/api/*) pour les donnees historiques InfluxDB.

3.1.1. Technologies frontend

Librairie	Role
Tailwind CSS (CDN)	Styles utilitaires, themes dark/light
MQTT.js	Connexion WebSocket vers Mosquitto :9001
Chart.js	Graphiques temps reel (sparklines, historique)
Three.js r128	Vue 3D de l'usine avec heatmap thermique
jsPDF 2.5.1	Export rapport PDF
jspdf-autotable 3.6.0	Tables dans les PDF exportes

3.2. Pages et fonctionnalites

3.2.1. Dashboard principal

Affichage en temps reel de :

- **KPIs globaux** : capteurs actifs, points/seconde, nombre d'alertes totales
- **Grille de capteurs** : cartes avec valeur live, sparkline, barre de progression, badge anomalie IA, statut colore (vert/orange/rouge)
- **Journal d'evenements** : log chronologique des connexions, alertes et systeme
- **Compteur energie et CO2** : consommation estimee en kWh et emissions CO2 depuis le debut de session
- **Filtre par zone** : Salle Serveur / Atelier / Local Electrique / Tout
- **Recherche** par nom de capteur ou machine

3.2.2. Score de sante predictif

Chaque machine dispose d'un **score de sante [0-100]** calcule en temps reel :

Score	Couleur	Interpretation
70 – 100	Emeraude	Bon etat, fonctionnement normal

40 – 69	Ambre	Degradation detectee, surveillance recommandee
0 – 39	Rouge	Etat critique – intervention preventive requise

3.2.3. Detection d'anomalies par Z-score (IA)

Calcul de **Z-score glissant** sur l'historique de chaque capteur (40 derniers points). Un badge « **XX% IA** » (violet) apparait lorsque l'ecart a la moyenne est statistiquement significatif.

```
z = |valeur_actuelle - moyenne| / ecart-type
badge = min(99, round(z x 28)) [affiche si z >= 1.25]
```

3.2.4. Vue 3D de l'usine

La page **Vue 3D** (Three.js r128) affiche :

- Modele 3D avec les 3 zones colorees (bleu, violet, ambre)
- 14 machines avec **heatmap thermique** dynamique (couleur temperature normalisee)
- Ventilateurs animes en rotation sur les machines concernees
- Lumieres d'avertissement pulsantes (rouge) en cas d'alerte critique
- Survol (tooltip) avec nom et statut de la machine
- Controles OrbitControls (rotation, zoom, panoramique)

3.2.5. Cartographie SVG (plan usine)

Page **Cartographie** : plan 2D de l'usine en SVG genere dynamiquement :

- 3 zones delimitrees avec couleurs de statut
- 14 machines positionnees avec indicateur colore
- Legende avec statut et comptage des capteurs par machine

3.2.6. Comparaison multi-machines

Page **Comparaison** : selection et affichage cote a cote de **2 a 4 machines** :

- Selecteur visuel avec indicateur de statut par machine
- Grille : score de sante, tous les capteurs avec barres de progression et badges IA
- Bouton « Retirer » pour deselectioner une machine

3.2.7. Alertes avancees

Page **Alertes** – historique filtrable de toutes les alertes :

- Filtres par severite (critique / avertissement / info), zone et machine
- Accuse de reception individuel (« Acquitter ») et global (« Tout acquitter »)
- Badge en temps reel dans la navigation (alertes non acquittees)
- Suppression des alertes acquittees (« Purger acquittees »)

3.2.8. Page Maintenance

Generation automatique :

- Tableau recapitulatif de l'etat de toutes les machines (score, statut, capteur le plus critique)

- Actions de maintenance recommandees par priorite (CRITIQUE / AVERTISSEMENT / OK)
- Top 3 des capteurs les plus anormaux (Z-score)

3.2.9. Export PDF

Rapport complet incluant :

- Resume global (capteurs actifs, alertes, energie, CO2, uptime)
- Tableau de tous les capteurs avec valeur actuelle et statut
- Score de sante par machine
- Rapport d'anomalies IA

3.2.10. Apparence et themes

- **Mode sombre / clair / systeme** (persiste en localStorage)
- **6 themes de couleur** : Azure, Emeraude, Violet, Rouge, Ambre, Cyan

3.2.11. Mode Kiosque

Defilement automatique entre les pages principales toutes les 30 secondes. Ideal pour affichage mural dans l'usine.

3.2.12. Modal d'accueil (onboarding)

Au premier chargement de chaque session, un modal presente les fonctionnalites principales :

- « **Commencer** » : ferme le modal pour la session
- « **Ne plus afficher** » : ferme definitivement (localStorage)

3.3. Architecture backend (server.js)

3.3.1. Routes API

Route	Me-thode	Description
/api/status	GET	Statut de tous les services
/api/sensors/latest	GET	Dernieres valeurs capteurs depuis InfluxDB
/api/history/:zone/:machine/:capteur	GET	Historique d'un capteur
/api/alerts	GET	Alertes recentes depuis InfluxDB
/api/stats	GET	Statistiques globales
/documentation.pdf	GET	Documentation technique (PDF statique)

/*	GET	SPA fallback -> index.html
----	-----	----------------------------

3.3.2. Securite

- **Helmet.js** – Headers HTTP (CSP, X-Frame-Options, HSTS)
- **CORS restraint** – origines autorisees via variable `CORS_ORIGINS`
- **Rate Limiting** – 120 req/min/IP sur toutes les routes `/api/*`
- **Validation des parametres** – whitelist regex avant injection dans requetes Flux

4. Composants de la plateforme

4.1. Mosquitto – Broker MQTT

4.1.1. Role

Mosquitto 2.0 est le broker central. Il expose trois listeners :

Port	Protocole	Auth	Usage
1883	MQTT	Anonyme	Docker interne (Node-RED, simulateur)
8883	MQTT/TLS	user + pwd	Clients externes securises
9001	WebSocket	Anonyme	Webapp (MQTT.js direct)

4.1.2. Configuration

```
listener 1883 0.0.0.0
protocol mqtt
allow_anonymous true

listener 8883 0.0.0.0
protocol mqtt
cafile /mosquitto/config/ca.crt
certfile /mosquitto/config/mosquitto.crt
keyfile /mosquitto/config/mosquitto.key
require_certificate false
allow_anonymous false
password_file /mosquitto/config/passwd

listener 9001 0.0.0.0
protocol websockets
allow_anonymous true

persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
log_type all
```



Attention : Le fichier `passwd` doit etre present avant le demarrage du conteneur. Il est genere par `make certs` avec les comptes `admin:admin123` et `sensor:sensor123`.

4.2. Node-RED – Traitement des flux

4.2.1. Architecture des flows

Onglet	Role	Noeuds principaux
Ingestion	Collecte MQTT -> InfluxDB	mqtt-in, function, http request
Automation	Regles metier + alertes	mqtt-in, switch, function, mqtt-out
Simulation	Tests internes	inject, function, mqtt-out

4.2.2. Onglet Automation – Regles metier

Regle	Capteur	Condition	Action
R1	Vibration	> 5.0 mm/s	Alerte debug + log InfluxDB
R2	Temperature	> 40 deg C	Avertissement debug
R2b	Temperature	> 80 deg C	Alerte critique + log
R3	Vibration	> 8.0 mm/s	Commande STOP machine
R5	Luminosite	< 50 lux	Alerte anomalie + log InfluxDB

4.3. InfluxDB 2 – Base de donnees temporelle

4.3.1. Parametres de configuration

Parametre	Valeur
Organisation	industritech
Bucket principal	iot_data
Token admin	mytoken123superlong
Utilisateur admin	admin

Mot de passe	admin123!
Port	8086

4.3.2. Schema de la measurement machine_data

machine_data

tags:

```

zone      = salle_serveur | atelier | local_elec
machine   = rack_principal | rack_secondeaire | onduleur | climatiseur
           | presse | robot | convoyeur | tour_cnc | compresseur
           | poste_soudure | armoire_1 | armoire_2
           | transformateur | groupe_electrogene
capteur    = temperature | humidite | vibration | courant | vitesse
           | luminosite | pression | puissance | debit | niveau_sonore
unite      = degC | % | mm/s | A | m/s | lux | bar | kW | L/min | dB

```

fields:

```

valeur    (float)    -- valeur mesuree
seuil     (float)    -- seuil de declenchement alerte
derive    (float)    -- derive progressive accumulee
anomalie  (integer)  -- 0 ou 1

```

timestamp: nanoseconde (precision=ns)

5. Simulateur de capteurs

5.1. Description

Le simulateur Python (`simulator/sensor_simulator.py`) emule **43 capteurs physiques** repartis sur **14 machines** dans les 3 zones de l'usine.

5.2. Zone Salle Serveur – 10 capteurs, 4 machines

Machine	Capteur	Unite	Base	Seuil	In-terv.
rack_principal	temperature	deg C	22.0	28.0	5s
rack_principal	humidite	%	45.0	70.0	5s
rack_secondaire	temperature	deg C	23.0	28.0	5s
rack_secondaire	humidite	%	42.0	70.0	5s
rack_secondaire	puissance	kW	2.8	5.0	5s
onduleur	temperature	deg C	25.0	35.0	8s
onduleur	puissance	kW	4.5	8.0	8s
onduleur	courant	A	12.0	20.0	8s
climatiseur	temperature	deg C	18.0	25.0	10s
climatiseur	debit	L/min	15.0	5.0*	10s

* anomalie si valeur < seuil

5.3. Zone Atelier – 19 capteurs, 6 machines

Machine	Capteur	Unite	Base	Seuil	In-terv.
presse	vibration	mm/s	2.0	5.0	2s
presse	temperature	deg C	30.0	40.0	2s
presse	pression	bar	180.0	250.0	2s
robot	courant	A	8.0	15.0	2s
robot	vibration	mm/s	1.0	4.0	2s
robot	temperature	deg C	28.0	38.0	3s
convoyeur	vitesse	m/s	1.0	0.2*	2s
convoyeur	vibration	mm/s	0.8	3.0	3s
tour_cnc	vibration	mm/s	1.5	6.0	2s
tour_cnc	temperature	deg C	32.0	45.0	3s
tour_cnc	courant	A	10.0	18.0	3s

tour_cnc	niveau_sonore	dB	72.0	90.0	3s
compresseur	pression	bar	8.0	12.0	4s
compresseur	temperature	deg C	35.0	50.0	4s
compresseur	vibration	mm/s	1.2	5.0	4s
compresseur	debit	L/min	80.0	30.0*	4s
poste_soudure	courant	A	120.0	200.0	3s
poste_soudure	temperature	deg C	40.0	55.0	3s
poste_soudure	luminosite	lux	800.0	1500.0	5s

* anomalie si valeur < seuil

5.4. Zone Local Electrique – 14 capteurs, 4 machines

Machine	Capteur	Unite	Base	Seuil	In- terv.
armoire_1	luminosite	lux	200.0	50.0*	10s
armoire_1	temperature	deg C	25.0	35.0	10s
armoire_1	courant	A	30.0	50.0	10s
armoire_2	temperature	deg C	26.0	35.0	10s
armoire_2	courant	A	25.0	45.0	10s
armoire_2	puissance	kW	18.0	30.0	10s
transformateur	temperature	deg C	40.0	65.0	6s
transformateur	puissance	kW	45.0	80.0	6s
transformateur	vibration	mm/s	0.5	2.0	6s
transformateur	niveau_sonore	dB	55.0	75.0	8s
groupe_electrogene	temperature	deg C	30.0	50.0	8s
groupe_electrogene	vibration	mm/s	1.8	5.0	8s
groupe_electrogene	debit	L/min	2.0	0.5*	8s
groupe_electrogene	niveau_sonore	dB	65.0	85.0	8s

* anomalie si valeur < seuil

5.5. Format du payload MQTT

```
{
  "zone":      "atelier",
  "machine":   "presse",
  "capteur":   "vibration",
  "valeur":    3.147,
```

```
"unite":      "mm/s",  
"seuil":      5.0,  
"anomalie":   false,  
"derive":     0.05,  
"timestamp":  "2026-03-13T09:30:00Z"  
}
```

Topic MQTT : factory/{zone}/{machine}/{capteur}

6. Securite

6.1. Certificats TLS

```
make certs          # genere certs + fichier passwd
```

Cree dans mosquitto/config/ :

- ca.key / ca.crt – cle privée et certificat CA (2048 bits RSA, 10 ans)
- mosquitto.key / mosquitto.crt – cle et certificat broker (5 ans, signe par CA)
- passwd – mots de passe Mosquitto (admin:admin123, sensor:sensor123)

6.2. Authentification

Service	Utilisateur	Credential	Portee
Mosquitto TLS	admin	admin123	Administration
Mosquitto TLS	sensor	sensor123	Publication capteurs
InfluxDB	admin	admin123!	Administration
InfluxDB API	(token)	mytoken123superlong	Any lecture/écriture
Grafana	admin	admin	Interface web
Node-RED	(secret)	iotsecret2026	Credentials chiffrés

6.3. Réseau Docker

Tous les services communiquent via le reseau bridge `iot-network`. Aucun trafic inter-services ne transite par l'interface hôte.

7. Docker Compose


```

services:
  mosquitto:
    image: eclipse-mosquitto:2.0
    container_name: iot-mosquitto
    ports: ["1883:1883", "8883:8883", "9001:9001"]
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  node-red:
    image: nodered/node-red:3.1
    container_name: iot-nodered
    ports: ["1880:1880"]
    environment:
      - TZ=Europe/Paris
      - NODE_RED_CREDENTIAL_SECRET=iotsecret2026
    depends_on: [mosquitto, influxdb]

  influxdb:
    image: influxdb:2.7
    container_name: iot-influxdb
    ports: ["8086:8086"]
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=admin
      - DOCKER_INFLUXDB_INIT_PASSWORD=admin123!
      - DOCKER_INFLUXDB_INIT_ORG=indusitritech
      - DOCKER_INFLUXDB_INIT_BUCKET=iot_data
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=mytoken123superlong

  grafana:
    image: grafana/grafana:10.2.0
    container_name: iot-grafana
    ports: ["3000:3000"]
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin

  sensor-simulator:
    build: { context: ./simulator }
    container_name: iot-sensor-sim
    environment:
      - MQTT_HOST=mosquitto
      - MQTT_PORT=1883
      - MQTT_USER=sensor
      - MQTT_PASS=sensor123
    depends_on: [mosquitto]

  webapp:
    build: { context: ./webapp }
    container_name: iot-webapp
    ports: ["8080:8080"]
    environment:
      - INFLUX_URL=http://influxdb:8086
    depends_on: [influxdb, mosquitto]

```

```
networks:
  iot-network:
    driver: bridge
    name: iot-network
```


8. Makefile – Reference des commandes

Commande	Description
<code>make init</code>	Genere les certificats TLS + structure de dossiers
<code>make certs</code>	Genere uniquement les certificats TLS et le fichier passwd
<code>make start</code>	Demarre tous les services
<code>make stop</code>	Arrete tous les services
<code>make restart</code>	Redemarre tous les services
<code>make status</code>	Affiche l'etat des conteneurs Docker
<code>make logs</code>	Logs en temps reel de tous les services
<code>make logs-mosquitto</code>	Logs Mosquitto uniquement
<code>make logs-nodered</code>	Logs Node-RED uniquement
<code>make logs-sim</code>	Logs du simulateur capteurs
<code>make mqtt-sub</code>	Ecoute tous les topics <code>factory/#</code>
<code>make mqtt-pub-test</code>	Publie un message de test MQTT
<code>make influxdb-query</code>	Requete Flux de test sur 5 dernieres minutes
<code>make test</code>	Execute la suite de tests d'integration (18 tests)
<code>make clean</code>	Supprime toutes les donnees persistees (volumes)
<code>make clean-all</code>	Supprime donnees + certificats

9. Tests d'intégration

N	Test	Methode
1	iot-mosquitto est running	docker ps filter
2	iot-nodered est running	docker ps filter
3	iot-influxdb est running	docker ps filter
4	iot-grafana est running	docker ps filter
5	Mosquitto MQTT :1883 acces- sible	netcat -z -w3
6	Mosquitto MQTT-TLS :8883 accessible	netcat -z -w3
7	Node-RED :1880 accessible	netcat -z -w3
8	InfluxDB :8086 accessible	netcat -z -w3
9	Grafana :3000 accessible	netcat -z -w3
10	InfluxDB ping -> 204	curl /ping
11	InfluxDB health = pass	curl /health + JSON
12	Grafana API health -> 200	curl /api/health
13	MQTT publish reussi	mosquitto_pub dans le conteneur
14	Fichier passwd Mosquitto present	test -f -s
15	Certificat ca.crt present	test -f
16	Certificat mosquitto.crt present	test -f
17	Certificat mosquitto.key present	test -f
18	flows.json present	test -f



Succes : Resultat production : **18/18 PASS** – pipeline complet operationnel.

10. Exploitation et maintenance

10.1. Commandes de diagnostic

```
# Donnees InfluxDB en temps reel
curl -s "http://localhost:8086/api/v2/query?org=industritech" \
  -H "Authorization: Token mytoken123superlong" \
  -H "Content-Type: application/vnd.flux" \
  -d 'from(bucket:"iot_data") |> range(start: -5m)
    |> filter(fn:(r)=> r._measurement == "machine_data")
    |> group() |> count() |> sum(column:"_value")'

# Surveiller les messages MQTT
mosquitto_sub -h localhost -p 1883 -t "factory/#" -v

# Logs
make logs                # Tous les services
make logs-mosquitto      # MQTT
docker logs iot-nodered 2>&1 | grep -i error | tail -20
```

10.2. Procedures de maintenance

```
make stop                # Arret (donnees conservees)
make clean                # Arret + suppression des donnees
docker compose up -d --build webapp    # Redeploy webapp uniquement
docker compose up -d --build sensor-simulator # Redeploy simulateur
```

10.3. Guide de depannage

Symptome	Cause probable	Solution
Mosquitto ne démarre pas	« Duplicate password_file »	depends_on unique-ment dans listener 8883
Permission denied certs	chmod 600 trop restrictif	chmod 644 sur les .crt/.key
Grafana permission denied	Conteneur uid 472	sudo chown -R 472:472 grafana/data
InfluxDB unauthorized	Token invalide	Verifier mytoken123superlong
Simulateur connexion refusee	Mosquitto pas encore pret	depends_on deja configure – attendre
Webapp sans donnees	MQTT WS non connecte	Verifier port 9001 / Mosquitto démarre

Bouton webapp sans effet	Fonction hors scope	Exposer via <code>window.maFonction = maFonction</code>
-----------------------------	------------------------	--

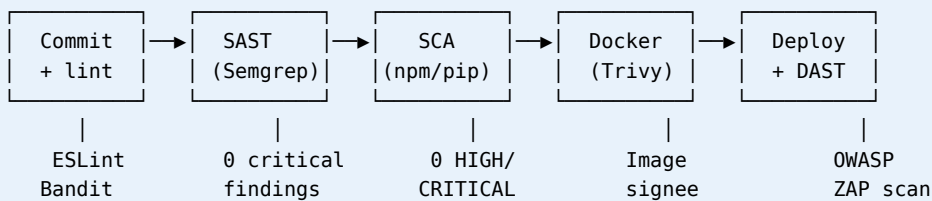
11. DevSecOps



Astuce : DevSecOps = Dev + Sec + Ops. La securite n'est pas une couche rajoutee apres coup : elle est **integree a chaque etape** du cycle de vie — du code source jusqu'au conteneur en production. Cette plateforme applique cette philosophie via des controles automatises, une surface d'attaque reduite et des secrets externalises.

11.1. Pipeline CI/CD securise

Le pipeline **deploye** (.github/workflows/devsecops.yml) integre des **security gates** bloquants en 7 etapes sequentielles :



Etape	Outil	Cible	Seuil bloquant
Lint / SAST	ESLint + Semgrep	Node.js / JS frontend	0 finding critique
SAST Python	Bandit	simulator/	Severity HIGH = fail
SCA Node	npm audit	webapp/ package.json	0 vuln HIGH/ CRITICAL
SCA Python	pip-audit	requirements.txt	0 CVE connu
Image scan	Trivy	Images Docker buildees	0 CRITICAL, max 5 HIGH
DAST	OWASP ZAP	http:// localhost:8080	0 alerte High
Secrets	Gitleaks	Repo complet	0 secret detecte

11.1.1. Fichiers DevSecOps implementes

Fichier	Role
---------	------

<code>.github/workflows/devsecops.yml</code>	Pipeline GitHub Actions 7 etapes (lint SAST SCA secrets image DAST policy)
<code>.pre-commit-config.yaml</code>	Hooks git pre-commit : Gitleaks, Bandit, ESLint, Hadolint, Trivy
<code>.eslintrc.security.json</code>	Regles ESLint plugin:security (injection, eval, regex unsafe)
<code>simulator/.bandit</code>	Config Bandit SAST Python – severity LOW+
<code>.gitleaks.toml</code>	Regles custom : token InfluxDB, mot de passe MQTT, secret Node-RED
<code>.hadolint.yaml</code>	Lint Dockerfile – seuil warning, registries de confiance
<code>.zap-rules.tsv</code>	Regle OWASP ZAP baseline (IGNORE/WARN par ID)
<code>security/policies/devsecops.rego</code>	Policy OPA/Rego – 7 controles bloquants avant deploy
Makefile (targets Dev-SecOps)	lint sast audit secrets-scan scan-images sbom policy devsecops

i Note : Conteneurs durcis : multi-stage build, utilisateur non-root appuser (UID 1001), HEALTHCHECK integre, `npm ci --omit=dev --ignore-scripts` pour install deterministe.

11.2. Analyse des dependances (SCA)

11.2.1. Webapp Node.js – npm audit



Succes : Resultat audit (14 mars 2026) : **0 vulnerabilite** detectee. 7 dependances directes, toutes a jour.

Paquet	Version	Role securite
express	^4.18.2	Framework HTTP – maintenu activement
helmet	^7.1.0	Headers securite (CSP, HSTS, X-Frame)

express-rate-limit	^7.2.0	Protection brute-force / DDoS
cors	^2.8.5	Controle origines cross-origin
mqtt	^4.3.7	Client MQTT (scenario demo uniquement)
axios	^1.6.2	Requetes HTTP vers InfluxDB
morgan	^1.10.0	Journalisation acces HTTP

11.2.2. Simulateur Python – pip-audit



Succes : paho-mqtt==1.6.1 : **0 CVE** reference. Seule dependance Python du simulateur.

11.3. SAST – Analyse statique du code

11.3.1. Regles ESLint (webapp)

Regles de securite actives dans le backend Express :

Controle	Implementation actuelle
Injection (SQL/Flux)	Whitelist regex VALID_ZONES, VALID_CAPTEURS, RE_MACHINE
Validation des entrees	validateParams() bloquant sur toute route /api/history
Rate limiting	120 req/min/IP via express-rate-limit
Headers HTTP	helmet() : CSP, X-Frame-Options: DENY, HSTS, nosniff
Eval / Function()	Absents – code vanilla sans eval dynamique
Secrets en clair	Aucun dans le code – variables d'environnement uniquement

11.3.2. Bandit (Python simulator)

```
bandit -r simulator/ -ll 2>&1
# >> No issues identified. (severity >= LOW, confidence >= LOW)
```

11.4. Scan d'image Docker (Trivy)

Integre dans le pipeline CI (image-scan job) et disponible via `make scan-images` :

```
# Installation Trivy
curl -sfl https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh

# Scan de l'image webapp
trivy image --severity HIGH,CRITICAL --exit-code 1 iot-monitoring-webapp:latest

# Scan du simulateur
trivy image --severity HIGH,CRITICAL --exit-code 1 iot-sensor-sim:latest
```

Les images cibles utilisent des bases **distroless-like** (node:20-alpine, python:3.11-slim) qui minimisent la surface OS :

Image	Base OS	Strategie de reduction
iot-monitoring-webapp	node:20-alpine	Alpine Linux – moins de 20 paquets OS
iot-sensor-sim	python:3.11-slim	Debian slim – deps minimales
eclipse-mosquitto:2.0	Alpine	Image officielle verifiee
influxdb:2.7	Debian slim	Image officielle verifiee
grafana:10.2.0	Ubuntu minimal	Image officielle verifiee

11.5. Conformite OWASP IoT Top 10

Application des recommandations OWASP IoT Top 10 (2024) :

No	Risque OWASP IoT	Statut	Mesure implementee
I1	Mots de passe faibles par default	OK	Credentials dans .env – non committes

I2	Services reseau in-utiles	OK	Seuls les ports necessaires exposes
I3	Interfaces non securisees	PARTIEL	TLS sur :8883 – Web-Socket :9001 sans TLS local
I4	Mecanismes de MAJ absents	OK	Images versionnees, rebuild CI/CD
I5	Composants obsoletes	OK	SCA npm + pip-audit en CI, 0 CVE connu
I6	Protection vie privee insuffisante	OK	Donnees telemetrie industrielle non personnelles
I7	Transferts de donnees non securises	PARTIEL	TLS sur MQTT externe – HTTP interne Docker
I8	Gestion des secrets deficiente	OK	.env hors Git, variables d'env Docker
I9	Systeme securise par default absent	OK	Helmet.js, rate-limit, CSP actifs par default
I10	Absence de durcissement physique	N/A	Environnement logiciel uniquement

11.6. Gestion des secrets



Attention : Le fichier `.env` contenant les credentials **ne doit jamais etre commite**. Il est reference dans `.gitignore` et fourni via `.env.example` comme modele.

Flux de gestion des secrets :

```
.env.example (commite -- valeurs placeholder)
|
▼ copie manuelle + remplacement
.env (non commite -- credentials reels)
|
▼ docker compose --env-file .env up
Variables d'environnement des conteneurs
|
▼ process.env.INFLUX_TOKEN etc.
Code applicatif (aucun secret en clair)
```

Pour un deploiement production, les secrets sont injectes via un gestionnaire dedie :

Outil	Usage recommande
-------	------------------

HashiCorp Vault		Injection de secrets au runtime via agent sidecar
Docker Secrets (Swarm)		Montage en fichier dans /run/secrets/
GitHub Secrets	Actions	Variables chiffrées injectées dans le pipeline CI
Azure Key Vault		Integration native ACI / AKS pour déploiement cloud

11.7. Matrice de menaces (STRIDE)

Application du modèle **STRIDE** sur le vecteur MQTT – pipeline critique :

Me-nace	Scenario	Impact	Contre-mesure
Spoofing	Client usurpe un capteur légitime sur :1883	Données falsifiées dans InfluxDB	ACL par topic, auth sur :8883
Tampering	Modification payload MQTT en transit	Fausse alerte ou silence d'alarme	TLS :8883, signature HMAC (evolution)
Reputation	Absence de log d'audit des publications	Impossibilité de tracer l'origine	Morgan HTTP logs + log_type Mosquitto
Info Discl.	Ecoute passive sur :9001 non chiffré	Espionnage des valeurs capteurs	TLS WebSocket (evolution)
DoS	Flood MQTT sur :1883	Saturation broker – perte données	Rate-limit Mosquitto max_inflight_messages
Elevation	Injection Flux via /api/history non valide	Exfiltration InfluxDB	Whitelist stricte + rate-limit API

11.8. Surface d'attaque reseau

Port	Service	Expose hote	Recommandation prod
1883	Mosquit-to MQTT	Oui – interne	Restreindre au CIDR interne uniquement
8883	Mosquit-to TLS	Oui	Conserver – TLS obligatoire
9001	MQTT WebS-cket	Oui	Passer en WSS (TLS) en production
1880	Node-RED UI	Oui	Fermer en production – admin only via VPN
3000	Grafana	Oui	Derriere reverse proxy Nginx + auth LDAP/OIDC
8080	Webapp	Oui	Derriere Nginx + HTTPS (Let's Encrypt)
8086	InfluxDB	Oui	Fermer vers l'exterieur – acces interne uniquement

12. Annexes

12.1. Recapitulatif des versions

Composant	Version	Source
Mosquitto	2.0	eclipse-mosquitto:2.0
Node-RED	3.1	nodered/node-red:3.1
InfluxDB	2.7	influxdb:2.7
Grafana	10.2.0	grafana/grafana:10.2.0
Python (simulateur)	3.11-slim	python:3.11-slim
Node.js (webapp)	20-alpine	node:20-alpine
paho-mqtt	1.6.1	pip
Three.js	r128	CDN unpkg
Chart.js	4.x	CDN jsdelivr
jsPDF	2.5.1	CDN jsdelivr
Typst	0.14.2	Compilateur documentation

12.2. Evolutions possibles

Axe	Description
Securite avancee	ACL Mosquitto par topic, JWT Node-RED, RBAC InfluxDB
Haute disponibilite	InfluxDB Cluster, Mosquitto bridge, reverse proxy Nginx
Alerting	Grafana Alerting + notifications Slack/SMTP
Machine Learning	Anomalies par LSTM / IsolationForest sur series temporelles
Retention policies	Downsampling Flux (5min pour +7j, 1h pour +30j)
MQTT 5.0	User Properties et Reason Codes
Edge computing	Deploiement Raspberry Pi / NUC avec images ARM

Capteurs reels	Integration capteurs physiques (Modbus, OPC-UA, BLE)
----------------	--



Documentation Technique v2.1 — Plateforme IoT IndustriTech
Hackathon 2026 · Module 10 · 14 mars 2026